

Lista de Exercícios — C++ Aula 5

Arrays unidimensionais e multidimensionais

1. Relatório de Diagnóstico de Sensores

O robô RollerBot possui 6 sensores de contato em sua base para detectar colisões. Crie um programa que leia do usuário o estado de cada sensor (considere 1 para sensor ativo e 0 para inativo). Armazene esses valores em um array e, ao final, imprima um relatório tabular informando o estado de cada um.

Saída esperada (Entrada: 1 0 1 1 0 0)

```
--- Diagnóstico RollerBot ---
Sensor [0]: ATIVO
Sensor [1]: INATIVO
Sensor [2]: ATIVO
Sensor [3]: ATIVO
Sensor [4]: INATIVO
Sensor [5]: INATIVO
-----
```

2. Sensor de Proximidade e Varredura

O Lidar do Way2Go realiza uma varredura em N ângulos fixos (Com N sendo um input do usuário, e os ângulos indo de 0° a 180°). O robô deve identificar o objeto mais próximo para evitar uma colisão iminente. Leia as N distâncias inseridas pelo usuário, armazene-as em um array e informe qual foi a menor distância capturada e em qual índice (posição da varredura) e o ângulo ela se encontra.

Considere que o índice 0 representa 0° e o índice N-1 representa 180°

Saída esperada (Entrada: 5 80.5 120.0 15.2 40.0 200.5)

```
Varredura Completa.
Menor distância detectada: 15.20 cm
Índice do obstáculo: 2
Ângulo de saída: 90°
```

3. Processamento de Ponto Crítico Térmico

Utilize as matrizes de teste da Câmera Térmica (5x5). O programa deve percorrer a matriz escolhida para encontrar o ponto de maior temperatura (Ponto Crítico). Imprima o valor da temperatura máxima encontrada e suas coordenadas (Linha e Coluna). Caso a temperatura seja superior a 80°C, exiba obrigatoriamente um alerta de segurança.

Saída esperada (Utilizando o Caso 1)

Análise Térmica Concluída.
Temperatura Máxima: 88.50°C
Localização: Pixel [2][2]
STATUS: [ALERTA DE SUPERAQUECIMENTO]

Matrizes a serem utilizadas:

```
float termica_isolada[5][5] = {  
    {20.5, 21.0, 22.2, 21.0, 20.8},  
    {21.2, 25.0, 30.5, 24.8, 21.5},  
    {22.0, 31.2, 88.5, 30.1, 22.1},  
    {21.5, 24.5, 30.2, 25.0, 21.0},  
    {20.8, 21.2, 22.0, 21.5, 20.5}  
};
```

```
float termica_quente[5][5] = {  
    {82.1, 85.0, 81.2, 84.5, 83.0},  
    {80.5, 92.1, 88.4, 90.2, 81.5},  
    {84.2, 89.5, 95.0, 88.1, 85.3},  
    {81.0, 87.2, 89.4, 86.5, 82.2},  
    {83.5, 84.1, 82.0, 85.3, 84.0}  
};
```

```
float termica_frio[5][5] = {  
    {15.2, 16.5, 15.8, 17.1, 16.0},  
    {14.8, 15.5, 16.2, 15.9, 15.1},  
    {16.1, 15.9, 17.5, 16.3, 15.8},  
    {15.5, 16.0, 15.2, 15.7, 16.2},  
    {16.3, 15.1, 16.4, 15.5, 15.9}  
};
```

4. Navegação em Grades de Ocupação

Utilize um dos Mapas de Ocupação fornecidos (7x7, 8x8 ou 10x10). O usuário deverá informar uma coordenada de destino [linha] e [coluna]. O programa deve realizar as seguintes validações:

- Checagem de Colisão: Verifique se a célula informada está ocupada (1) ou livre (0). Se estiver ocupada, informe “Destino Inválido: Obstáculo Detectado”.
- Busca de Proximidade: Caso a célula esteja livre, o robô deve identificar o obstáculo mais próximo daquela posição. Percorra toda a matriz, calcule a distância de cada obstáculo encontrado até a coordenada digitada e

informe a localização do obstáculo mais próximo.

Exemplo de saída para o mapa A (Entrada: 5 5):

Verificando coordenada [5, 5]...

Status: Caminho Livre.

Buscando obstáculo mais próximo...

Obstáculo mais próximo encontrado em: [4, 3]

Distância Euclidiana: 2.24

Matrizes a serem utilizadas

```
int mapa_esparso_A[7][7] = {
    {0, 1, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 1, 0, 0},
    {0, 0, 0, 0, 0, 0, 0},
    {1, 0, 0, 0, 0, 0, 1},
    {0, 0, 0, 1, 0, 0, 0},
    {0, 0, 0, 0, 0, 0, 0},
    {0, 0, 1, 0, 0, 0, 0}
};
```

```
int mapa_esparso_B[8][8] = {
    {0, 0, 0, 0, 0, 0, 0, 0},
    {0, 1, 1, 0, 0, 0, 0, 0},
    {0, 1, 1, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 0, 0, 0},
    {0, 0, 1, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 1, 1, 1},
    {0, 0, 0, 0, 0, 1, 1, 1},
    {0, 0, 0, 0, 0, 0, 0, 0}
};
```

```
int mapa_esparso_C[10][10] = {
    {0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    {0, 1, 1, 0, 0, 0, 0, 1, 0, 0},
    {0, 1, 1, 0, 0, 1, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 0, 0, 0, 0, 1},
    {0, 0, 0, 0, 1, 0, 0, 0, 0, 0},
    {0, 1, 0, 0, 0, 0, 0, 1, 1, 0},
    {0, 0, 0, 0, 0, 0, 0, 1, 1, 0},
    {0, 0, 1, 1, 1, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 0, 0, 0, 1, 0},
    {0, 1, 0, 0, 0, 1, 0, 0, 0, 0}
};
```